VISIUM HD PIPELINE

From sample sheets to reviewed cell objects

Operator and maintainer handbook

v5.1rc1 + scoped metadata-refresh fix
November 4, 2026

A source-grounded guide to metadata, pipeline stages, scientific workers, storage, reports and the Python modules behind the Project Center.

Documentation only. No installation, new approval or production-code replacement is required.

Companion reference: open docs/visiumhd/README.md in the repository for the 100-file module catalog and versioned source references. This PDF is the offline reading copy.

How to use this handbook

Operators: begin with Chapters 1–5, then use Chapters 12 and 15 for analysis and handoff. Maintainers: continue through the stage explanations, then use the Markdown Module Reference to find every defined function and its source location.

1. The short orientation
2. Metadata has not been removed
3. Four locations to distinguish
4. The source layout: which folder owns what?
5. What happens when you click Run?
6. Pipeline stages, producers and checkpoints
7. Coordinates and image handling
8. H&E evidence and bin QC
9. Annotations, staining and review records
10. Shared StarDist prior, separate reconstruction methods
11. Cell QC, annotation transfer and master storage
12. Independent clustering and optional joint integration
13. Reports: where to look and who produces them
14. Resources, environments and external executables

15. Colleague handoff and routine operation
16. Troubleshooting by responsibility
17. Maintenance guide and extension boundaries
18. Current limits and interpretation of validation

Source references such as vhd/center/importers.py:95–171 refer to the exact audited snapshot. In the Markdown edition they open the versioned source snapshot. This PDF edition preserves the references as readable text.

Documented baseline: VisiumHD_Pipeline_v5_1_Dual_Method_20261104 (release 0.5.1rc1) plus scoped-metadata-refresh-20261104-v1.

Purpose: explain the implemented pipeline, its metadata inputs, its processing modules, and their connections. This is documentation and a set of illustrative input tables—not a pipeline upgrade, installer, new approval, or scientific validation. It does not change the user's existing code, configuration, masters, or results.

Reading paths. A new operator can read Sections 1–5, 12 and 15 first. A maintainer should then read the processing sections and open the companion **Module Reference**, which covers every Python file under vhd/, tools/, and scripts/. The **Source Browser** provides the exact documented source with line numbers; the source inventory records hashes so this handbook is not mistaken for documentation of an unknown future version.

1. The short orientation

This repository is no longer a set of independent analysis notebooks. It has three layers: the Project Center you operate, a controller that plans and records work, and scientific workers that read and write data. The main notebook is a launcher for that system. Most .py files are imported libraries or worker entry points, not scripts that colleagues should execute one after another.

The normal entry is `notebooks/00_Project_Center.ipynb`. It calls `launch_setup()`, which opens or creates a `ProjectCenter` and then displays the live widget interface. Opening a project does not run segmentation. View displays evidence, Save records a decision, and Run invokes computational work. The saved HTML report is a read-only view of the state, not an independent approval application. vhd/center/ui.py:363–435 vhd/center/controller.py:559–561 vhd/center/reports.py:66–67

Notebook	Role	Usual frequency
00_Project_Center.ipynb	Project setup, reviews, execution and reports	Routine use
01_Dual_Method_Target_Smoke.ipynb	Two tiny real-science Bin2cell/SpatialData checks in the selected worker environment	New environment or relevant changes
99_Project_Center_Environment_Check.ipynb	Environment inventory and small controller/method tests	Setup and troubleshooting

The smoke notebook does not test StarDist inference, actual Proseg inference, full-slide memory, or biological correctness. Its storage test is important even after installation: it checks that appending one reconstruction does not alter existing method tables. The patch documented here restricts metadata consolidation to the changed table and container root. vhd/persistence/spatial.py:28–64

The useful mental model is:

```
Your input sheets and optional profile
  -> Project Center's managed project
  -> Plan and explicit review decisions
  -> Selected Python worker / external Proseg executable
  -> Scientific products plus execution receipts
  -> Updated live status and read-only HTML reports
```

A status, a review and a successful test are different things. A file can exist without being accepted; a gallery can be reviewed before the actual source is imported; a tiny smoke test can pass while a full sample still needs local performance and biological review. The controller explicitly keeps recorded output inventory separate from newly verified execution. vhd/center/stages.py:85–94 vhd/center/controller.py:224–231

2. Metadata has not been removed

2.1 Three meanings of “metadata”

In this codebase, the same word refers to three distinct mechanisms. Keeping them separate resolves much of the apparent complexity.

Kind	Examples	Source and destination
Sample descriptors	Species, treatment, donor, cohort, timepoint, tumor measurements	Input sample sheet or joined metadata CSV; constant columns in each sample's bin/cell .obs
Spatially varying annotations or measurements	Tissue, Necrosis, Anthropotic, stain intensity	Asset sheet plus reviewed category choices; per-bin memberships or numerical values, optionally summarized to cells
Technical/provenance metadata	Source hashes, transforms, table-region links, completed-step signatures	Generated by the code and stored in JSON, .uns, or Zarr headers; not biological descriptors to type manually

Sample descriptors are handled by vhd/control/sample_metadata.py and vhd/sample_metadata.py. Spatial assets are handled by vhd/annotations/. Storage headers and selective updates are handled by vhd/persistence/. These are related but not interchangeable systems. vhd/control/sample_metadata.py:133–226 vhd/sample_metadata.py:113–124 vhd/annotations/workflow.py:263–360 vhd/persistence/spatial.py:72–137

2.2 Option A: metadata columns in the new-project sample sheet

In **Create a new project from sample and asset sheets**, extra descriptive columns become sample metadata. The example catalog already demonstrates this with species, treatment and donor_id; the standalone metadata example disappeared during consolidation, not the feature itself.

For example, a compact new-project sample sheet can contain:

```
sample_id,stage,goal,spaceranger_outs,segmentation_image,image_type,species,treatment,donor_id,timepoin
t
Sample01,raw,bin_QCed,/data/study/Sample01/outs,/data/study/images/Sample01.svs,svs,human,Vehicle,001,D
ay0
Sample02,raw,bin_QCed,/data/study/Sample02/outs,/data/study/images/Sample02.svs,svs,human,Treatment_A,0
02,Day7
```

The compact-sheet reader recognizes pipeline fields, handles species specially, and places other nonempty columns in each row's _sample_metadata. Text such as donor 001 remains text. These extra values are not automatically converted to floats or integers. Blank extra cells in this route are omitted from that sample's metadata mapping. vhd/center/importers.py:95–171

Scope matters: this flexible reader is used by the new-project sheet route. The older **Import existing pipeline configuration** route uses a strict full-schema sample manifest. Do not add arbitrary extra columns to a legacy manifest and assume both readers accept it. Keep the legacy metadata settings or use the new-project sheet route deliberately. vhd/control/manifest.py:162–190 vhd/center/controller.py:23–38

2.3 Option B: a separate collaborator metadata CSV

The new-project setup form still contains **Metadata CSV (optional)** and **Metadata ID column**. These controls are shown only for the new-project-sheet operation, not for opening an existing workspace or importing a legacy configuration. That UI visibility is one reason this feature is easy to miss. vhd/center/ui.py:363–435

A separate table can look like this:

```
Sample,species,treatment,donor_id,baseline_tumor_volume_mm3
Sample01,human,Vehicle,001,113.75
Sample02,human,Treatment_A,002,115.83
```

Set the optional metadata path to that CSV and **Metadata ID column** to `Sample`. The match uses exact sample IDs, not row order or directory names. Every enabled sample must match by default; unused extra metadata samples are allowed by default. Duplicate metadata keys and conflicting values fail rather than selecting an arbitrary first or last row. vhd/control/sample_metadata.py:133–226 vhd/control/sample_metadata.py:245–317

You may put some descriptors in the sample sheet and other descriptors in the separate CSV. Identical overlapping values are accepted; conflicting values are rejected in the compact setup path. For clarity, the provided companion-CSV example places the biological descriptors in only one source. vhd/center/importers.py:95–171

Current type behavior: the simple form's separate-CSV route uses the metadata reader's default `string` type. `113.75` is therefore text in that setup, unless a different supported advanced configuration is used. A blank cell in a separate CSV is a missing value (`None`) by default. The literal `NA` is not silently converted to missing. IDs are matched exactly and cannot contain surrounding whitespace. vhd/control/sample_metadata.py:133–226

2.4 Option C: typed metadata in an existing full configuration

The full configuration loader retains a `sample_metadata` section in `settings.json`. It supports an explicit join key, selected columns, output renaming, and source-column types. A fragment—not a complete settings file—looks like this:

```
{
 "sample_metadata": {
  "path": "sample_metadata.csv",
  "sample_id_column": "Sample",
  "columns": ["species", "treatment", "donor_id", "baseline_tumor_volume_mm3"],
  "default_type": "string",
  "types": {"baseline_tumor_volume_mm3": "float"},
  "missing_values": [""],
  "require_all_samples": true,
  "allow_extra_samples": true
 }
}
```

Use this with **Import existing pipeline configuration**, after adding it to the actual full settings file. Type declarations and renames refer to the **source heading**, not the renamed output. Supported types are `string`, `int`, `float`, `bool` and opt-in `auto`. Composite dataset/sample keys and full sample-key matching are also supported by this loader. vhd/control/sample_metadata.py:133–226 vhd/control/manifest.py:198–272

Do not put this fragment in the new-project optional profile and assume its type settings will control the form's separate metadata input. In the inspected version, that path constructs a new metadata configuration containing only the selected path and ID column. This is a current interface limitation, not a recommendation to hand-edit managed files. vhd/center/importers.py:95–171

2.5 Where sample descriptors go

The path through the code is:

```
Sample extras or metadata CSV
  -> row['_sample_metadata']
  -> row_config(...): spec['metadata']
  -> attach_sample_metadata(adata, spec)
  -> adata.obs[descriptor] + adata.uns['vhd_sample_metadata_json']
```

Attachment occurs during bin/master and cell preparation/assembly, not necessarily in every raw intermediate file. String descriptors use one-category categorical columns per sample rather than millions of independent copies. Numerical and Boolean types are retained when they were supplied as such. The .uns JSON records ownership and source provenance. It does not replace expression or coordinates. vhd/control/manifest.py:315–350 vhd/sample_metadata.py:60–110 vhd/sample_metadata.py:113–124 vhd/adapters/assemble.py:45–135 vhd/reconstruction/storage.py:88–122

Fields such as qc_class, total_counts, region, instance_id, coordinate columns and protected prefixes cannot be used as metadata output names. Existing conflicting non-owned columns are not overwritten. Automatic metadata attachment is deliberately conservative. vhd/control/sample_metadata.py:43–50 vhd/sample_metadata.py:60–110

An advanced sync_sample_metadata() function can update sample descriptors in already assembled bins_2um, cells, proseg and bin2cell tables using selective .obs/.uns writes. It has explicit replacement/pruning options for columns owned by this metadata mechanism. This is not a general UI metadata editor and should not be used with an improvised sample specification. vhd/sample_metadata.py:127–180

2.6 Existing managed projects are not live CSV joins

The Project Center saves a resolved project in project_state.json, snapshots the original inputs, and creates managed configuration exports. Opening the same workspace reads that saved state. Editing the original metadata CSV afterward is not a supported live-update mechanism; editing files under engine/ can trigger the managed-configuration consistency guard. vhd/center/importers.py:37–92 vhd/center/controller.py:81–86 vhd/center/controller.py:61–62

For a new study, include descriptors during setup. For an already running project needing a metadata correction, first review the required controlled update; do not reimport into the same workspace, replace its state JSON, or rerun segmentation merely to add a descriptor. This release does not expose a general post-creation metadata-edit or bulk sheet-merge panel.

Read-only inspection from an opened notebook is simple:

```
center = setup["center"]
for row in center.project["rows"]:
    print(row["sample_key"], row.get("_sample_metadata", {}))
```

The sample overview also displays **Sample metadata and inputs**. The handbook examples are described in metadata_examples/README.md; they contain invented data and require your own input/output paths.

3. Four locations to distinguish

Location	Meaning	Edit policy
Code checkout	vhd/, tools/, scripts/, notebooks, tests, examples	Maintainer-controlled versioned software
Original setup sheets/profile	Your reproducible initial input definitions	Human-authored; preserved at Project Center creation
Controller workspace	Managed state, revisions, input snapshots, generated engine configuration, HTML	Use the Project Center; do not manually synchronize internal CSVs
Scientific work/master/publication locations	Large arrays, intermediates, masters, plots, durable products	Determined by project paths and storage policy; no automatic mount management

A typical controller workspace is:

```
ProjectCenterWorkspace/
  project_state.json        # authoritative controller snapshot
  .controller.lock          # advisory participating-controller lock
  revisions/                # prior state revisions
  inputs/                   # initial source snapshots
  engine/                   # managed compatibility exports
    samples.csv
    settings.json
    assets.csv
    asset_choices.csv
    overrides/
  html/
    project_review.html
    samples/<sample_key>/
      bin_qc.html
      assets.html
      segmentation.html
      cells.html
      clustering.html
```

The controller's own state is small and contains no expression arrays. It uses an advisory lock and same-directory replacement for JSON writes. Those mechanisms are not a declaration that every mounted object store is POSIX-compatible, nor do they detect unrelated notebooks that access the same biological master. vhd/center/state.py:61–115 vhd/center/state.py:46–58

The default compact-sheet setup organizes work, scratch, cache, masters and published products below an existing approved output root. An existing full configuration retains its separately specified locations. No username, project folder or storage mount is inferred as a universal rule. vhd/center/importers.py:95–171 vhd/control/paths_v3.py:41–45 vhd/control/manifest.py:274–305

4. The source layout: which folder owns what?

Location	Responsibility
vhd/center/	Live UI, managed project, routes, saved reviews, resources, HTML and independent sample analysis
vhd/control/	Full-manifest parsing, per-sample settings, stage dispatch, input checks, metadata CSVs and publication policy
vhd/compute/	Worker processes, isolated environment checks, thread/GPU setup, scVI and clustering execution
vhd/he/	Fixed-reference H&E measurement, bin-score sampling, classification and distribution reviews
vhd/annotations/	Asset discovery, alignment proposals, geometry review, approval and bin/cell metadata transfer
vhd/adapters/	Existing-bin/cell input adaptation and base SpatialData assembly
vhd/reconstruction/	Method selection, Bin2cell adapter, assignment boundaries, named-method storage and comparisons
vhd/persistence/	Selective writes, metadata refresh, transaction journals and explicit recovery
Root vhd/*.py	Shared numerical contracts, coordinates, images, I/O, QC, segmentation, Proseg cells, masters and joint analysis
tools/	Standalone diagnostics, preview/report helpers and release checks; not a required sequential workflow
scripts/run_pipeline.py	Optional lower-level CLI for selected legacy actions; not a full Project Center replacement
tests/	Regression tests, fixtures and dependency-gated scientific tests

The companion Module Reference has one entry per Python file, its defined functions/classes, observed internal imports, and observed importing files. Those relationships come from static parsing, including imports inside functions. They are **not** a claim that every imported function executes on every route, and dynamic runpy/subprocess dispatch can add relationships beyond static imports.

5. What happens when you click Run?

A normal UI action travels through the following chain:

```
notebooks/00_Project_Center.ipynb
  vhd.center.ui.launch_setup / show_center
    ProjectCenter.plan / run_until_review
      center.stages.route + reconstruction.config.apply_method_route
      center.resources.runtime_project
        compute.launch.launch_samples
          runtime_checks.preflight_python
          control.prechecks.check_action_inputs
          python -m vhd.compute.worker
            control.runner.run_action
              selected scientific stage
```

The parent process chooses the worker executable and constructs its environment before scientific imports. That separates the notebook UI from TensorFlow/PyTorch initialization and makes the selected worker environment—not merely the notebook kernel—the relevant dependency boundary. Per-stage workers write logs and resource reports. vhd/compute/launch.py:94–189 vhd/compute/worker.py:5–13 vhd/compute/runtime_checks.py:94–144

The controller selects ready jobs sharing the next action and can submit them as a batch subject to worker budgets. It pauses at unresolved review gates. The operation is synchronous in the notebook; it is not a detached scheduler. Interrupts and failed invocations are recorded, and a previously running job must be inspected before it is acknowledged as stopped. vhd/center/controller.py:470–535 vhd/center/controller.py:628–636

Completion requires more than a remembered notebook execution count. The scientific stages use input signatures and required products; the controller also records contracts and output stamps. A stale or partial product is not silently adopted. Neither deleting a marker nor editing a signature is a supported recovery. vhd/core.py:151–164 vhd/core.py:167–169 vhd/center/controller.py:224–231

CLI limitation: scripts/run_pipeline.py exposes its own explicit action list. In the documented release it does not expose all named-method or independent-clustering actions. Use the Project Center for the full dual-method route rather than assuming every new action is available through that older CLI.

6. Pipeline stages, producers and checkpoints

Starting material, resume point and endpoint are distinct. stage describes the supplied material, not the latest completed job. A source can remain raw while its managed progress is at annotation review. Resume selection checks prerequisites but does not certify skipped biological work. vhd/center/stages.py:32–57 vhd/center/controller.py:168–185

Stage/action	Main implementation	Main products or review
ingest_raw	io.ingest	bins_raw.h5ad, geometry.json, ingestion receipt
import_bins / reuse_bin_qc	adapters.bins	Prepared bins and accepted QC sidecar
measure_he	he.workflow.measure	Fixed H/E maps, bins_he.parquet, measurement record
bin_qc	he.workflow.classify_stage or legacy qc.build_bin_qc	bins_qc_obs.parquet, QC report and H&E overlay
review_bins	Project Center review gate	Saved coordinate and QC review; no automatic biological judgment
assemble_bins	bin_master.ensure_bin_master	Validated bin-master checkpoint
review_assets	center.controller + annotations helpers	Review gallery, usage, coverage and source/alignment evidence
transfer_assets	annotations.workflow.transfer_assets	Stored assets, annotated bin sidecar and transfer record
stardist / review_prior	segmentation.stardist_prior + controller	Shared QC-gated nucleus prior and review
proseg / postprocess_cells	segmentation.run_proseg / cells.build_cells	Durable Proseg outputs, cells, boundaries and QC
Optional bin2cell_secondary / bin2cell	reconstruction.bin2cell_adapter	Optional density objects; exact assignments and aggregated cells
assemble_methods	reconstruction.storage.assemble_methods	Independent method tables/shapes and comparison report
refresh_method_annotations	reconstruction.storage.refresh_method_annotations	Method-specific bin-derived cell metadata
import_cells / legacy assemble	adapters.cells / master	Standalone existing-cell route; legacy cells retained
review_cells	Project Center review gate	Inspect the selected populations before analysis
cluster_individual / review_clustering	center.single_sample + controller	Companion analysis object and ten resolution plots
Joint analysis phases	integration, compute	Separate opt-in scVI preparation/training/clustering/writeback
publish / export_existing	control.runner + control.storage	Controlled publication or preserved annotated-input export

The executable dispatch and route code are authoritative; some operations are controller-managed rather than cases in run_action(). Review gates are not worker action names. vhd/control/runner.py:63–150 vhd/center/stages.py:32–57 vhd/reconstruction/config.py:105–122

qc_bins stops at the reviewed bin master before optional assets. bins can include optional reviewed asset transfer. Cell-only starting material cannot manufacture a bin endpoint. An export-only route preserves a supplied annotated object instead of pretending to regenerate upstream results. vhd/center/stages.py:32–57 vhd/control/runner.py:40–60

7. Coordinates and image handling

7.1 Coordinate systems are not interchangeable

The pipeline preserves the original per-bin full-resolution pixel coordinates as the source reference. Array row/column coordinates identify the physical bin grid; downsampled hires/lowres images have their own pixel grids; segmentation crops and Proseg coordinates require their own transforms. A display thumbnail must not redefine the original bin coordinates. vhd/io.py:54–110 vhd/core.py:253–265 vhd/compact_images.py:39–77

The grid-fitting path validates the delivered bin positions against an affine model and, where necessary, a projective model. It does not round fractional coordinates to force an affine fit. A projective model is represented by explicitly transformed polygon vertices, not passed to SpatialData as if it were an affine raster transform. vhd/grid_geometry.py:192–261 vhd/projective_grid.py:94–146 vhd/projective_storage.py:29–56

Numerical agreement with a position table is not independent anatomical registration. The coordinate report and the bin/H&E overlay serve different purposes. Projective bin footprints are model-derived geometry; their final GeoDataFrame is eager even though polygon construction is batched. vhd/grid_diagnostics.py:72–99 vhd/projective_storage.py:29–56

7.2 Compact master images

The compact image policy uses native Space Ranger hires/lowres images and their calibration. The shared resolver accepts complete same-folder pairs and the reviewed split layout where images and scale JSON are in different supported folders under the same configured `outs`. It does not borrow an image from another sample or infer registration from matching filenames. vhd/spaceranger_images.py:130–184 vhd/compact_images.py:9–35

Original image paths and transforms remain recorded for segmentation and provenance; compact display images do not require duplicating the full original whole-slide image in the master. Explicit source-image overrides remain configuration decisions. vhd/compact_images.py:39–77 vhd/he/source.py:15–39

8. H&E evidence and bin QC

The quantitative route uses fixed-reference optical-density deconvolution, clips negative concentrations, scales H and E by fixed divisors, combines the image channels, and samples the combined field at bin centers. The score is tissue evidence, not a calibrated tissue probability or a mass measurement. The supplied fixed divisors are H 1.9705 and E 1.0308; weighting and other effective settings are recorded. vhd/he/fixed.py:60–75 vhd/he/fixed.py:99–105 vhd/he/workflow.py:11–47

For a soft-union combination, the calculation is 1 - (1 - Hw) * (1 - Ew) after clipping weighted fixed-scale channels into [0,1]. The packaged backend defaults use H/E weights of 1.0/1.0; imported projects may have different explicit settings. Display-oriented Macenko normalization in he/display_legacy.py is separate and should not be substituted as the quantitative tissue-score definition. vhd/he/fixed.py:78–96 vhd/he/display_legacy.py:67–339

Measurement and classification are separate. The classifier requires an explicit tissue-score threshold and transcript threshold. With transcript_transform="identity", the transcript threshold is a raw count per bin, even when a plot uses logarithmic spacing. Both comparisons use >=. The fixed-score route keeps both tissue-high classes; the transcript cutoff labels high/low transcript bins but is not an independent retention requirement under this policy. Invalid measurements become unassessed. vhd/he/workflow.py:50–64

Distribution review reads existing measurement/observation columns rather than the expression matrix. It produces the tissue-score histogram, counts histogram, reproducible scatter subset and all-valid-bin density summary. Display transforms do not modify counts or QC thresholds. vhd/he/qc_review.py:503–582 vhd/he/qc_review.py:125–238 vhd/count_display.py:5–9

The bin review joins three observations: spatial alignment, sensible evidence distributions and the resulting classification map. The controller can save cutoffs before classification, then save the coordinate/QC review afterward. It refuses to overwrite an already completed QC revision with a different cutoff automatically; a complete branching/revision UI is not implemented in this release. vhd/center/controller.py:293–312 vhd/center/controller.py:314–341

9. Annotations, staining and review records

9.1 Independent categories, not one exclusive class

An annotation file can contain overlapping Tissue, Necrosis, artifact and tumor categories. The geometry reader keeps their identities; membership is evaluated independently. A bin may belong to more than one category. Negative regions are subtracted within their parent annotation, not treated as positive tissue or merged across categories. vhd/annotations/geometry.py:11–52 vhd/annotations/geometry.py:71–92

An invalid individual polygon is different from overlap between two valid polygons. The optional inspection/review/adoption modules inventory invalid regions, display candidates and require explicit acceptance before a derived geometry is activated. Accepted polygons retain all pieces and holes; reviewed standalone line remnants are not converted into invented area. The original XML remains unchanged. vhd/annotations/region_inspection.py:197–249 vhd/annotations/reviewed_geometry.py:267–303 vhd/annotations/reviewed_geometry.py:342–390

9.2 Preview first, scientific use second

The Visual-First helper plots the currently configured geometry and native signal on H&E without building a master or changing approvals. center.controller integrates it with discovery, selection edits and the visual-approval bridge, so the operator should use the live panel rather than the historical sequence of manual CSV edits and separate approval cells. tools/preview_assets_on_he.py:523–658 vhd/center/controller.py:372–377 vhd/center/controller.py:379–438

Saving asset usage binds the selected report evidence to the current sources, transforms and requested outputs. It does not declare a source physically stored. Actual transfer verifies/imports required sources before writing metadata. This distinction is encoded in visual_approval.ensure_transfer_sources(). vhd/annotations/visual_approval.py:245–430 vhd/annotations/visual_approval.py:464–509

9.3 What the choice fields mean

Choice	Meaning
include_in_obs	Store the selected measurement as bin metadata
obs_name	Safe output identifier; the category's original name is separate
qc_use	include, exclude or none; not a Boolean field
coverage_mode	Interpret unmarked regions as unknown, assessed negatives, or negatives within a separate assessed-area polygon
signal_methods	Selected numerical bin summaries such as mean, median, sum, min, max or explicit center sampling
cell_methods	Optional requested summaries of bin metadata at the cell level
same_extent_confirmed	Reviewed same-field-of-view/orientation assumption for dimension-based image alignment

The compiler rejects unsafe output names and collisions with core fields. An unselected row is not activated simply because it still has a suggested name. positive_only means outside polygons is unassessed; it does not invert a mask. exhaustive makes unmarked positions negative over the assessed domain. polygon supplies a separate coverage boundary. Positive-only exclusions are not accepted as if outside regions were known clean. vhd/annotations/discovery.py:166–212 vhd/annotations/geometry.py:71–92

For auto_extent, dimensions propose a transform; a scale hint checks that proposal. Same extent means corresponding physical field of view, not equal image dimensions. Crops or different fields require a suitable explicit

transform/landmarks. Signal value interpretation—binary codes versus native continuous values—is a separate decision from spatial scale. vhd/annotations/alignment.py:50–93 vhd/annotations/workflow.py:142–149

9.4 From imported geometry to per-bin and per-cell values

Polygon measurements use bin-center membership and explicit coverage. Continuous image summaries use the declared bin footprint and overlap with source pixels, including projective quadrilaterals where appropriate. Means and medians are overlap-weighted; sums are fractional-source-pixel sums, not automatically positive-pixel counts or molecule counts. Out-of-bounds/missing is not silently equated with a measured zero. vhd/annotations/workflow.py:263–360 vhd/annotations/area_sampling.py:28–103 vhd/annotations/projective_sampling.py:13–82

The annotation-based segmentation decision is added alongside the base QC. It combines the union of selected inclusion categories and the union of selected exclusions with three-valued assessment logic. The base classification is retained. Whether all excluded bins are also removed from Proseg input is controlled separately by `filter_proseg_bins`; do not assume filtering the nucleus prior and filtering every input transcript/bin are the same operation. vhd/annotations/transfer.py:55–84 vhd/reconstruction/config.py:98–102 vhd/segmentation.py:186–264

10. Shared StarDist prior, separate reconstruction methods

The shared H&E StarDist stage reads the reviewed image/crop geometry, runs the selected model, and gates nuclei using eligible bins and optional pathology rules. The packaged primary defaults are `2D_versatile_he`, probability `0.02`, NMS `0.30`, and target MPP `0.3`. Imported explicit settings take precedence; these values are a selected profile, not universal optimization results. vhd/segmentation.py:65–160 vhd/center/controller.py:659–702

The prior has its own signature, labels, metadata and QC image. Proseg and Bin2cell consume that prepared prior. A changed completed prior is not silently overwritten because a UI control was changed. vhd/segmentation.py:56–62 vhd/center/controller.py:659–702

10.1 Proseg

`segmentation.run_proseg()` validates the executable and arguments, prepares inputs from eligible bin counts and prior assignments, launches the external process, and verifies expected outputs. This executable is separate from the Python environment; the configured executable invocation matters. `cells.build_cells()` reads durable Proseg results, reconciles feature IDs, converts geometry into the established coordinate frame, computes QC, and produces prepared cells/boundaries. vhd/segmentation.py:170–183 vhd/segmentation.py:186–264 vhd/cells.py:176–349

10.2 Bin2cell

`reconstruction.bin2cell_adapter` uses the installed official Bin2cell expansion/aggregation functions on the existing raw-bin object and shared prior. It does not invoke an alternative input reader or silently destripe/normalize stored raw counts. The adapter independently checks exact assignment sums and coordinate/identity contracts before writing cells. `max_bin_distance=2` is the packaged default in array-bin units; `0` is a supported no-expansion setting in the adapter. vhd/reconstruction/bin2cell_adapter.py:51–70 vhd/reconstruction/bin2cell_adapter.py:152–234 vhd/reconstruction/config.py:49–79

`reconstruction.geometry` traces assigned bin regions, preserves holes/disconnected parts, and transforms their vertices. It avoids creating one eager polygon per input bin merely to derive a cell boundary. This differs from the projective bin-master representation, whose final bin GeoDataFrame is eager. vhd/reconstruction/geometry.py:54–89 vhd/projective_storage.py:29–56

10.3 Optional transcript-density objects

Secondary segmentation is off by default and is a separate worker action when enabled. It forms a density image, uses a fluorescence-style StarDist model with separate parameters, and performs secondary-label salvage. H&E thresholds are not silently reused as the secondary thresholds. Its inference branch remains an additional target pilot beyond the two required tiny smoke tests. vhd/reconstruction/bin2cell_adapter.py:245–278 vhd/reconstruction/config.py:105–122

10.4 Method names and downstream selection

Generated-cell projects with explicit `cells.methods` use `proseg` and/or `bin2cell` tables. Old configurations without that key retain the legacy `cells` route. Imported standalone cells remain `cells`; filenames do not establish their reconstruction method. The analysis method is selected explicitly from generated methods. vhd/reconstruction/config.py:18–27 vhd/reconstruction/config.py:34–46

Proseg keeps its original prepared-cache layout; Bin2cell uses `work/bin2cell/` and `reports/bin2cell/`. Each method receives separately qualified cell IDs, boundary relationships, receipts and reports. Running the second method must not delete or rewrite the first method's unrelated data. vhd/reconstruction/config.py:82–95 vhd/reconstruction/storage.py:63–75 vhd/reconstruction/storage.py:88–122

11. Cell QC, annotation transfer and master storage

11.1 Cell QC is not analysis QC

Prepared cells have technical metrics and an initial qc_pass decision. The later analysis-QC stage uses declared raw counts and a separate filter before feature selection, normalization and clustering. A storage table can therefore retain more cells than the analysis subset. This prevents a plotting/analysis choice from silently destroying the master count matrix. vhd/cells.py:176–349 vhd/reconstruction/bin2cell_adapter.py:108–149 vhd/analysis_qc.py:81–119

Bin-derived cell annotations use different assignments for the two methods. Bin2cell can use its exact discrete bin assignment. Proseg keeps the existing geometry-based bin summaries, which are not claims about molecule assignments. Boolean summaries include valid-bin counts and coverage; majority results can be positive, negative, mixed or missing. Categorical ties are explicit. Numerical summaries preserve missingness rather than turning an entirely unassessed cell into zero. vhd/reconstruction/storage.py:145–183 vhd/annotations/transfer.py:6–52

11.2 Master contents

A dual-method master can hold:

```
images:  he_hires, he_lowres, optional H/E maps and approved signals
shapes:  annotations, projective bins where needed,
      proseg_cell_boundaries, bin2cell_cell_boundaries
labels:  affine-grid bin labels / optional prior where applicable
tables:  bins_2um, proseg, bin2cell
      cells may also remain from a pre-existing legacy route
```

The exact keys depend on the actual route and selected sources. Named tables are linked to their own regions/instance IDs. Existing cells is not automatically renamed. Selective loading is provided by io.load_table() and load_spatial_with_table() so a caller need not load every expression table. vhd/reconstruction/storage.py:125–142 vhd/storage.py:64–71 vhd/io.py:133–155 vhd/io.py:158–163

11.3 Three levels of storage responsibility

bin_master.py establishes sample identity, tracks the bin-master checkpoint and checks prerequisites. adapters/assemble.py, master.py and reconstruction/storage.py decide which biological elements should exist. persistence/spatial.py encodes/writes selected elements, while persistence/transaction.py handles recoverable component replacement. These responsibilities are intentionally separate. vhd/bin_master.py:226–278 vhd/master.py:64–114 vhd/persistence/spatial.py:140–216 vhd/persistence/transaction.py:68–105

With the scoped-refresh fix, refresh_metadata(store) consolidates only the root. refresh_metadata(store, table_names=("bin2cell",)) also refreshes that changed table, then the root. Unrelated table roots are not independently reconsolidated. This prevents the metadata-only mutation that the target storage smoke test caught. It does not change any scientific algorithm or weaken preservation checks. vhd/persistence/spatial.py:28–64

Component replacements are journaled and retain recovery payloads; they are not atomic transactions across all Zarr keys. Single-writer acknowledgment and participating locks are safeguards, not permission to concurrently use another notebook against the same mutable store. Recovery is explicit; do not delete partial stores, backups or lock files solely because a stage failed. vhd/persistence/transaction.py:68–105 vhd/persistence/transaction.py:108–127

12. Independent clustering and optional joint integration

The independent route uses the explicitly selected cell table, raw-count analysis QC, gene detection filtering, in-memory normalization/log1p, HVG/PCA, neighbors and UMAP. Leiden resolutions 0.1 through 1.0 share one embedding. It writes a companion embedding/metadata object with zero genes rather than replacing the master's RNA matrix. The route is CPU Scanpy, not singleton scVI batch correction. vhd/center/single_sample.py:11–95

When no mitochondrial threshold is supplied and analysis QC is enabled, the code uses species metadata (or an explicit analysis species setting): human 20%, mouse 10%. It uses a strict < max_pct_mt retention comparison. Missing recognized mitochondrial features triggers an error unless an explicit skip policy was chosen; missing features do not prove a true zero fraction. The default minimum counts/cell is 3, minimum genes/cell 1, and minimum cells/gene 3. vhd/analysis_qc.py:13–30 vhd/analysis_qc.py:33–62 vhd/analysis_qc.py:65–69

Joint analysis is opt-in. Group configuration requires explicit members and a compatible selected table; methods are not mixed silently. integration.prepare_scvi() builds the common feature/count input; compute.scvi_worker trains and infers; compute.clustering handles the configured graph/clustering backend. The inspected scVI worker uses batch_key='sample'. Adding a treatment column as metadata does not automatically make treatment a corrected covariate. vhd/center/controller.py:574–588 vhd/control/runner.py:183–209 vhd/integration.py:47–169 vhd/compute/scvi_worker.py:13–91

The documented worker implementation makes multi-GPU training an explicit mode, not an inference from the number of visible GPUs. Training and latent inference are separate; inference is single-process/single-writer. The stored training-state checks distinguish a reusable saved model from complete inference and avoid blindly overwriting a model. vhd/compute/launch.py:191–196 vhd/compute/scvi_state.py:16–47 vhd/compute/scvi_worker.py:13–91

palette.py provides the recorded microViz-derived categorical colors and labels any extensions beyond its base colors. Plotting colors and count-axis transforms are presentation helpers, not changes to biological values. vhd/palette.py:27–60 vhd/count_display.py:21–25

13. Reports: where to look and who produces them

The fixed entry is <workspace>/html/project_review.html. Per-sample HTML pages live below html/samples/<sample_key>/; group reports live below html/groups/<group>/. They collect known report patterns and small summaries. They do not open expression matrices or native TIFFs just to render the index. Bounded image thumbnails and bounded CSV/JSON excerpts keep the summaries usable. vhd/center/reports.py:34–53 vhd/center/reports.py:70–101 vhd/center/reports.py:104–153

Review question	Evidence to look for	Producer
Are bins geometrically consistent?	geometry.json, grid diagnostic	grid_diagnostics, projective_grid
Are tissue/count cutoffs sensible?	Fixed H/E/score maps, cutoff histograms/scatter/density	he.workflow, he.qc_review
Do bin positions and QC match H&E?	bins_qc_class.png or imported-QC overlay, bin_qc_report.json	H&E QC or existing-bin adapter
Do all categories/signals align?	assets_on_he_*/index.html, summary.json, per-category images	Visual-First helper, asset navigation
What was accepted in an invalid polygon?	Repair gallery and derived geometry audit	region_review, reviewed_geometry
What will be used for segmentation?	Selected usage and segmentation_keep provenance	discovery compiler and annotation transfer
Is the nucleus prior plausible?	stardist_prior_ROI.png, audit, prior_metadata.json	segmentation
What differs between methods?	reconstruction_comparison.html and method QC	reconstruction.reports
Did a worker actually finish?	worker_<action>.log, resource report, status_<action>.json	compute.launch, control.runner
Which cells/clusters were analyzed?	Individual/group companion and UMAP/cluster-size reports	independent or joint analysis modules

The Project Center updates at action boundaries and explicit refresh. It does not install a background watcher. A historical report is evidence to inspect, not automatic proof that a newly configured action succeeded. The static HTML has no active approval controls; use the connected interface for Save and Run. vhd/center/controller.py:105–115 vhd/center/controller.py:121–128 vhd/center/reports.py:66–67

14. Resources, environments and external executables

The implementation separates the notebook interface from worker roles: spatial, StarDist, scVI and RAPIDS Python invocation paths. They may point to the same tested UV environment or to separate environments. Runtime checks probe the configured interpreter; they do not silently install missing packages. Executable paths preserve virtual-environment invocation semantics rather than resolving a Python symlink to a different environment.
vhd/compute/runtime_checks.py:16–33 vhd/compute/runtime_checks.py:36–60 vhd/compute/launch.py:41–55

The Project Center resource probe records CPU affinity, visible cgroup limits, RAM headroom and NVIDIA information, then applies conservative optional caps. Its observation is not a resource reservation. Unknown per-job RAM keeps a pilot constraint; imported memory estimates remain user-configured assumptions unless actually measured. CPU budgets, Dask scratch locations and GPU-per-sample work are different from distributed single-model training.
vhd/center/resources.py:67–85 vhd/center/resources.py:88–125 vhd/compute/launch.py:22–39

Sharing the tested UV project is useful, but the handoff should also identify the expected worker executables, selected Proseg binary/version, and the Jupyter frontend setup used for the live controls. The pipeline's external Proseg preflight and widget/frontend checks exist because a Python dependency list alone is not the whole runtime contract. This handbook does not prescribe a new UV lock or update Proseg. vhd/compute/startup_support.py:315–323 vhd/segmentation.py:170–183 vhd/center/ui.py:363–435

15. Colleague handoff and routine operation

Give colleagues the patched source, their input sheets/profile, the documented environment, the external Proseg installation details where needed, and this handbook. The smoke notebook and test files should stay in the distribution even when they are not routine per-sample notebooks. Do not redistribute personal source snapshots, internal sample paths, raw data or approval records unless intended and authorized.

For a colleague's first environment, the practical order is environment check, tiny scientific smoke tests, then Project Center. For a new study, select the appropriate sample/asset examples, include sample metadata, create a new controller workspace, inspect paths and start with one sample. For an existing project, reopen its existing workspace instead of repeatedly importing it. These are operating recommendations; the code checks the actual selected route and stored state, not this document's reading order.

Before running, review the printed plan, worker executable and storage destinations. After a review gate, save the decision in the corresponding panel. Acknowledge write access separately from scientific acceptance. A successful Save does not mean the transfer/reconstruction has executed, and a successful worker exit does not supply a biological review. vhd/center/controller.py:266–291 vhd/center/controller.py:314–341 vhd/center/controller.py:343–353

Sharing an already tested environment does not eliminate the value of a colleague's tiny read/write smoke test on their actual filesystem. It does reduce avoidable package mismatches. Keep the expected zero-skip result and the installed scoped metadata fix together with the distribution; do not accidentally re-share the unpatched original release ZIP.

16. Troubleshooting by responsibility

Symptom	First place to inspect	Avoid
VBox(...) or IntSlider(...) text only	Jupyter widget frontend/environment; ui.launch_setup guidance	Editing biological inputs
Project cannot open after manual CSV edits	project_state.json revision/history and managed hashes	Copying another CSV over engine/
Sample metadata missing	Setup mode; _sample_metadata; join report; attachment stage	Assuming every raw intermediate already has final .obs
Wrong metadata type	Inline/default-string route versus explicit typed legacy configuration	Assuming numeric-looking text was inferred
Source/registration changed	Discovery/visual approval evidence and actual transform/source	Typing approval signatures manually
Signal binary encoding mismatch	Native signal values and declared interpretation	Silent thresholding or multiplying values for display
Shape invalid	Region diagnostic and candidate review	Removing legitimate category overlaps
Master identity or partial-output refusal	bin_master, method receipts and transaction journal	Deleting/renaming master elements to force reuse
Existing table fingerprint changed	Scoped metadata-refresh patch and external writers	Weakening preservation assertions
Missing executable/package	Selected worker probe log, not only notebook imports	Installing unrelated packages into the wrong environment
No eligible ready action	Plan's review/setting requirement, endpoint and resume choice	Running all historical notebooks blindly
Cluster results from wrong reconstruction	cells.analysis_method, companion source-table record	Combining duplicate reconstructions as independent cells

Use the full worker log and current project configuration when reporting a failure. Include the action, sample key, relevant source file/line, software environment and small receipts. Large biological objects are usually not the first diagnostic artifact needed. Status and report location helpers are preferable to guessing a path from a historical run.
vhd/control/manifest.py:307–313 vhd/center/stages.py:85–94 vhd/compute/launch.py:57–92

17. Maintenance guide and extension boundaries

A maintainer should first identify the responsible layer rather than add a second independent route. UI text and controls belong in center/ui.py; stateful decisions and invalidation belong in center/controller.py; route modifications belong in center/stages.py and the method route helper; actual algorithms belong in their scientific modules; storage changes belong in the assembly/persistence modules. Tests should target both the small numerical contract and the end-to-end stage connection.

For a new worker action, check its route, worker-role/import requirements, preflight, dispatch, output inventory, controller contract, report registration, review gates, and reuse behavior. Adding only a function and a button can reproduce the earlier disconnect where a gallery existed but could not be approved or consumed. The current source makes these connections explicit. vhd/compute/runtime_checks.py:36–60 vhd/control/runner.py:63–150 vhd/center/stages.py:60–82 vhd/center/controller.py:187–222

For another cell-reconstruction method, preserve raw-count semantics, shared coordinate identity, method-specific IDs/regions, independent receipts and selected analysis table. Named-method storage is deliberately append-oriented. Changing an existing method's result in-place is not the same operation as adding another independent method. vhd/reconstruction/storage.py:88–122 vhd/reconstruction/config.py:105–122

The code still retains compatibility entry points and duplicate standalone geometry-review helpers. The Module Reference labels these rather than implying they are all steps the operator should run. tools/test_review_annotation_candidates.py is a test module despite living under tools/. he/display_legacy.py is a display-normalization implementation, not the fixed evidence computation. annotations/review.py supports legacy file-level approvals; the ordinary center route uses discovery and visual approval.

18. Current limits and interpretation of validation

This handbook documents the shipped source plus one delivered persistence patch. It is not a redesign specification. General post-creation metadata editing, bulk sheet reimport/merge, automatic scientific revision branching, detached execution and universal partial-store recovery are not implemented as one-click operations. The optional secondary segmentation requires its own pilot. Advanced coverage-polygon and legacy routes can have evidence requirements beyond the ordinary exhaustive-coverage gallery. Those limitations are not solved by documentation.

The user reported that the two required tiny target tests passed after applying the scoped metadata fix. This handbook records that as a user-reported environment result, not as a new execution of those scientific tests here. The documentation checks separately exercise metadata imports and verify the source index/examples; they do not establish segmentation accuracy, full-slide speed, source-image alignment or storage behavior on institutional mounts.

The source audit identifies the exact base archive and patch hashes and the resulting production-module hashes. Function and line references in this handbook resolve to that snapshot. Future code changes may invalidate these references; regenerate the catalog and review changed behavior before claiming that a future release is documented by this handbook.

Companion files

- **Module Reference:** all 86 `vhd` Python modules, 13 `tools` files and one CLI script; functions, class methods and static relationships.
- **Source Browser:** read-only line-numbered code for that documented snapshot, plus selected configuration examples.
- **Metadata examples:** inline and separate-CSV routes, a typed-settings fragment for the full configuration path, and a blank companion template.
- **Source audit and validation:** exact provenance, inspection scope, example/contract checks and known limits.

No installation is required to read this documentation. No pipeline source is replaced by opening or extracting it.